

Evaluating and Optimising AI Assistants

Learning from conversations: logging, feedback, evaluation, and improving the assistant one measured change at a time

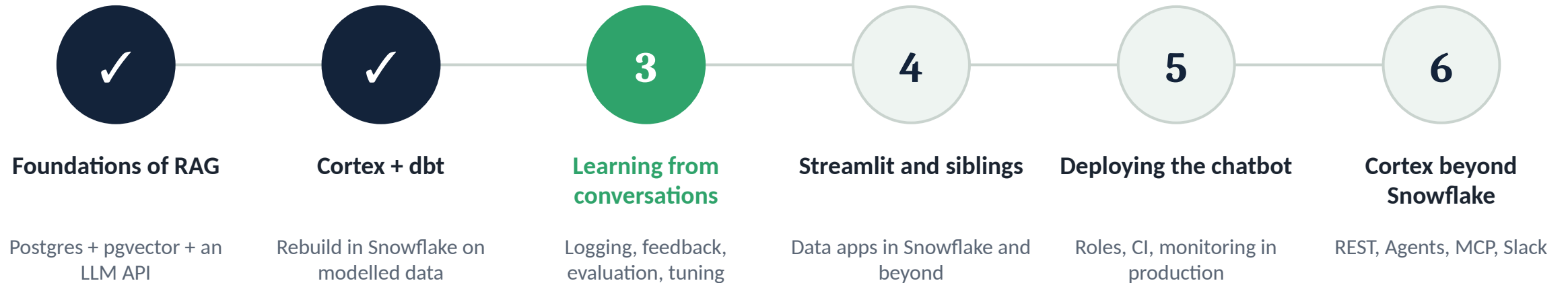
R. Firman Insan Muhammad (Vim)

SnowPro Certified • Data Engineering and AI Trainer

Scenario: Neo Nexus, a loyalty assistant for Caffè Nero. Illustrative only: all data, documents and figures are synthetic.



From a working assistant to an improving one



Today's output: conversation logs, an evaluation set, an automated judge, and one measured improvement with before and after numbers.

By the end of today you can...



Instrument

an assistant so every turn is logged with its route, retrieved context, model, prompt version, cost and feedback.



Evaluate

retrieval, answers and SQL with a curated test set and a calibrated LLM judge, all in SQL.



Diagnose

failures from real conversations and trace each one to the layer that caused it.



Optimise

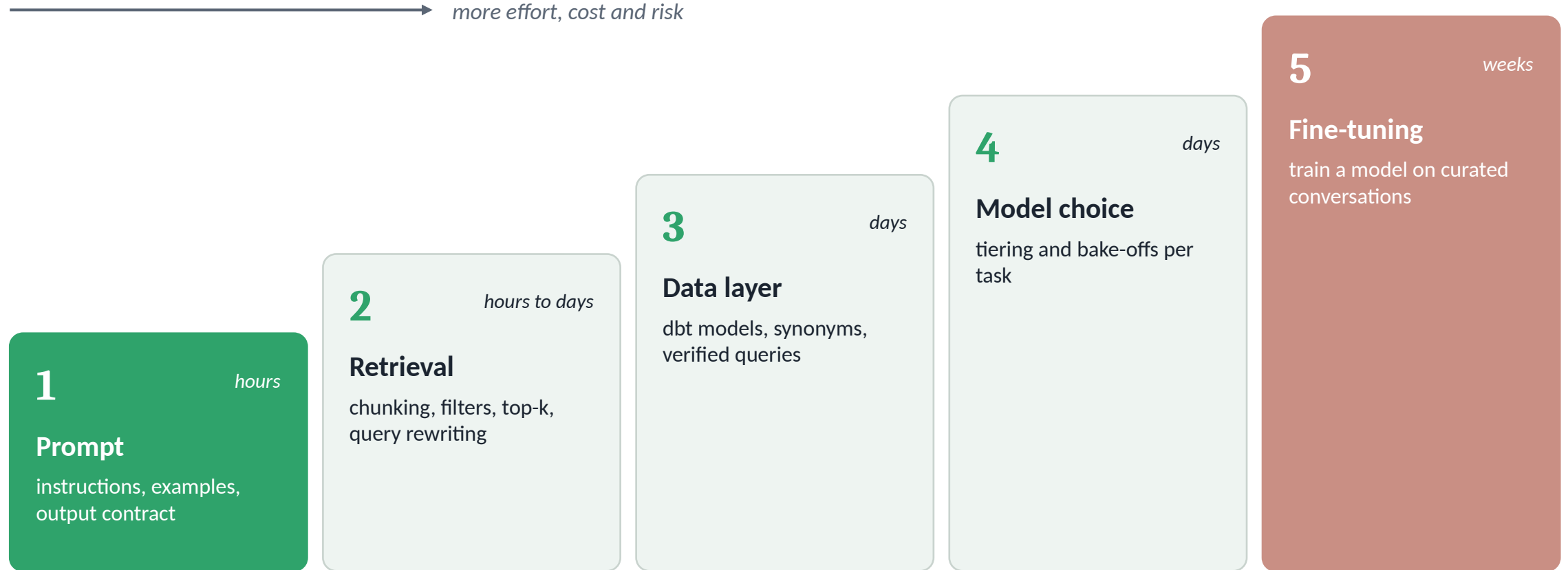
with the cheapest effective lever first, and ship only changes that pass a regression gate.

"Training the chatbot" rarely means training the model

Meaning	What changes	Who does it	Use it for
Pre-training	The base model's weights, from scratch	Model vendor	<i>Never your job</i>
Fine-tuning	Weights, on your prompt and answer pairs	You, occasionally	Style, format, distilling to a smaller model
System improvement	Data, retrieval, prompts, routing, model choice	You, continuously	Knowledge, accuracy, freshness, cost

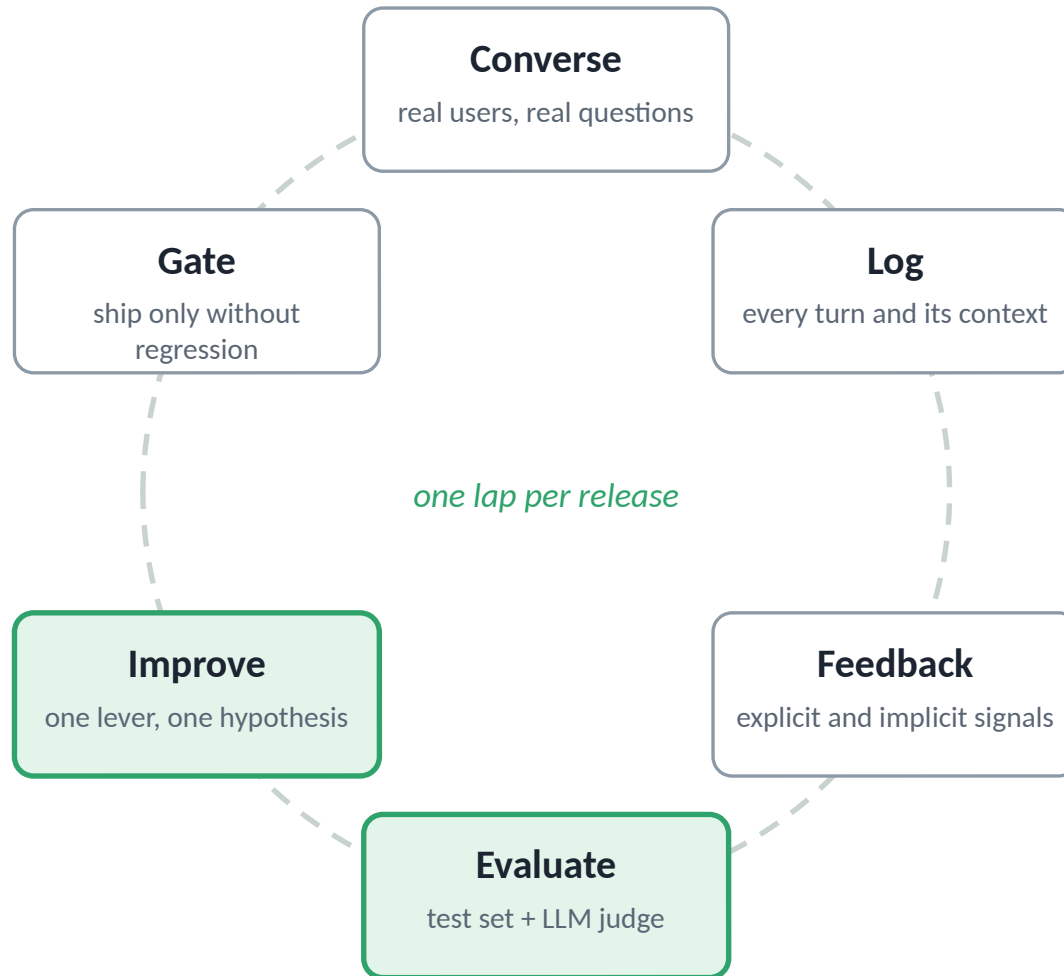
Rule of thumb: if the assistant does not know something, fix the data or retrieval. If it knows but answers badly, fix the prompt. Fine-tune only when a cheaper model must behave like an expensive one.

Climb from the cheapest lever to the most expensive



Start at rung 1. Climb only when the evidence says the lower rung cannot fix it.

Conversations become evidence; evidence becomes changes



Part 1 • Capture

Log and Feedback: what happened in each turn, and was it good?

Part 2 • Evaluate

Evaluate: a repeatable score for the current version, and a map of its failures.

Part 3 • Optimise

Improve and Gate: pull the cheapest effective lever and prove it helped.

01

Capture

Log every turn with its context, and collect the signals that tell you whether it went well

Log enough to replay and diagnose any turn

Request

question, standalone question, session, user role, timestamp

Route

DOCUMENT or ANALYTICAL, and the router's model

Retrieval

chunk IDs and scores, filters applied

Generation

model, prompt version, tokens in and out, latency

Output

answer text, generated SQL, abstained or not

Feedback

thumbs, correction text, follow-up behaviour



Logs are customer data. Apply masking policies to free text, set a retention period, and restrict who can read them.

Two tables in Snowflake: turns and feedback

SNOWFLAKE SQL

```
create table obs.chat_turns (  
  turn_id      string default uuid_string(),  
  session_id   string,  
  user_role    string,  
  asked_at     timestamp_ltz default current_timestamp(),  
  question     string,  
  standalone_q string,  
  route        string,          -- DOCUMENT | ANALYTICAL  
  chunk_ids    array,  
  chunk_scores array,  
  generated_sql string,  
  model        string,  
  prompt_version string,  
  input_tokens  number,  
  output_tokens number,  
  latency_ms    number,  
  answer        string  
);  
  
create table obs.feedback (  
  turn_id  string,  
  rating   number,          -- +1 / -1  
  correction string,  
  created_at timestamp_ltz default current_timestamp()  
);
```

Append-only

one row per turn; never updated

Arrays

chunk IDs and scores stay with the turn

Separate feedback

arrives later, joined on turn_id

Modelled in dbt

stg_chat_turns feeds evaluation

One function wraps every turn

PYTHON • SNOWPARK

```
def chat(session_id, history, message):
    t0 = time.perf_counter()
    q = standalone(history, message)          # Part 3: multi-turn
    route = classify(q)                       # DOCUMENT | ANALYTICAL
    if route == "DOCUMENT":
        answer, hits = answer_docs(q); sql = None
    else:
        answer, sql = answer_analytical(q); hits = []

    session.sql("""
        insert into obs.chat_turns
        (session_id, user_role, question, standalone_q, route,
         chunk_ids, generated_sql, model, prompt_version,
         latency_ms, answer)
        select ?, current_role(), ?, ?, ?, parse_json(?),
               ?, ?, ?, ?, ?""",
        params=[session_id, message, q, route,
                json.dumps([h["chunk_id"] for h in hits]), sql,
                MODEL, PROMPT_VERSION,
                int((time.perf_counter() - t0) * 1000), answer]
    ).collect()
    return answer
```

Build it yourself

Full control of what is captured; the tables join directly to marts and eval runs.

Or use the managed option

Snowflake AI Observability traces Cortex apps; TruLens covers apps hosted elsewhere.

Most feedback is implicit: read behaviour, not just thumbs

Signal	Type	Usually means	Strength
Thumbs down	Explicit	the answer was wrong or unhelpful	Strong, but rare
Typed correction	Explicit	the right answer, in the user's words	Gold
Rephrase within a minute	Implicit	the first answer missed	Strong
Session ends after an abstain	Implicit	a coverage gap in the data	Medium
Escalation to a human	Implicit	out of scope or low confidence	Medium
Answer copied	Implicit	the answer was used	Weak positive

Treat implicit signals as review candidates, not ground truth.

02

Evaluate

A curated test set, metrics for every layer, and an LLM judge you have checked against people

Build the test set from four sources, not one



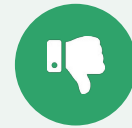
Golden set

Module 1's hand-written questions with expected chunks



Fresh eyes

the colleague questions from Module 2's homework



Mined from logs

thumbs-down and rephrased turns, reviewed and labelled



Adversarial

out of scope, prompt injection, requests for personal data

Stratify by route (document or analytical), channel, and difficulty, so regressions show up per slice.

Every fixed production failure becomes a permanent test case.

Measure every layer, so a failure points to its cause

Layer	Metric	How it is computed
Retrieval	Recall@5, MRR	expected chunk IDs vs retrieved chunk IDs
Answer	Faithfulness, relevance	LLM judge with a rubric, calibrated on human labels
Analytical SQL	Execution accuracy	generated SQL returns the same rows as the reference SQL
Behaviour	Correct abstain, correct refusal	adversarial slice, judged or rule-checked
Operations	p95 latency, cost per answer	from obs.chat_turns and Cortex usage views

Always report per slice. An average can improve while one channel gets worse.

Grade thousands of answers in one SQL statement

SNOWFLAKE SQL

```
create or replace table eval.judged as
select r.run_id, r.question_id,
       try_parse_json(ai_complete('claude-opus-5-5',
       'You grade a loyalty assistant. Use ONLY the ' ||
       'context. Return JSON: {"reasoning": string, ' ||
       '"faithfulness": 1-5, "relevance": 1-5, ' ||
       '"should_abstain": boolean}' ||
       '\n\nContext: ' || r.context ||
       '\n\nQuestion: ' || r.question ||
       '\n\nAnswer: ' || r.answer)) as verdict
from eval.run_results r;

select run_id,
       avg(verdict:faithfulness::number) as faithfulness,
       avg(verdict:relevance::number)   as relevance
from eval.judged
group by run_id;
```

Reason, then score

consistency improves when the judge explains first

Different model family

judges tend to favour their own style

Anchored 1 to 5 scale

describe what a 1, 3 and 5 look like

Calibrate on 30 human labels

trust the judge only once it agrees with people

Judge SQL by its results, not its text

SNOWFLAKE SQL

```
-- 1 when both queries return exactly the same rows
select iff(count(*) = 0, 1, 0) as exact_match
from (
  ( select * from table(result_scan(:generated_qid))
    minus
    select * from table(result_scan(:reference_qid)) )
  union all
  ( select * from table(result_scan(:reference_qid))
    minus
    select * from table(result_scan(:generated_qid)) )
);
```

Why not compare SQL text?

Different joins, aliases and CTEs can be equally correct.
Only the results matter to the user.

Normalise first

Align column order and names; round totals before comparing.

Store the reference SQL for every analytical test case. It doubles as a verified query candidate in Part 3.

Classify what went wrong, then fix the right layer

SNOWFLAKE SQL

```
select
  ai_classify(
    'Question: ' || t.question ||
    ' Answer: ' || t.answer ||
    ' Correction: ' || coalesce(f.correction, ''),
    ['retrieval miss', 'wrong SQL',
    'unsupported claim', 'out of scope',
    'formatting']
  ):labels[0]::string as failure_type,
  count(*) as turns
from obs.chat_turns t
join obs.feedback f using (turn_id)
where f.rating = -1
group by failure_type
order by turns desc;
```

Failure type	Fix at rung
Retrieval miss	2 • Retrieval
Wrong SQL	3 • Semantic layer
Unsupported claim	1 • Prompt
Out of scope	1 • Prompt / routing
Formatting	1 • Prompt, or 5 • Fine-tune

Read ten turns per class before acting: the classifier triages, people decide.

03

Optimise

Climb the ladder: prompt, retrieval, semantic layer, model choice, and fine-tuning only when earned

Version prompts like code, and test them like code

SNOWFLAKE SQL

```
create table app.prompts (
  prompt_version string,      -- answer_v7
  purpose        string,      -- answer | route
  template       string,
  is_active      boolean,
  created_by     string,
  created_at     timestamp_ltz
                default current_timestamp()
);

-- the version used is logged on every turn:
-- obs.chat_turns.prompt_version
```

PYTHON • A/B SPLIT

```
def pick_prompt(session_id):
    h = hashlib.sha1(session_id.encode())
    bucket = int(h.hexdigest(), 16) % 100
    return ("answer_v8" if bucket < 10
            else "answer_v7")    # 10% canary
```

Offline before online

Run both versions on the evaluation set; only A/B test the winner with users.

Conversations need standalone questions before retrieval

PYTHON • QUERY REWRITING

```
REWRITE = ("Rewrite the user's last message as a "
           "standalone question using the conversation. "
           "Return only the question.")

def standalone(history, message):
    convo = "\n".join(f"{t['role']}: {t['text']}"
                      for t in history[-6:])
    return complete("claude-haiku-4-5",
                    f"{REWRITE}\n\n{convo}\nuser: {message}",
                    session=session)
```

"And in the app?" → "Can members redeem a free drink in the app?"

Then tune one knob at a time

Chunk size and overlap

dbt model parameters

Attribute filters

channel, document version

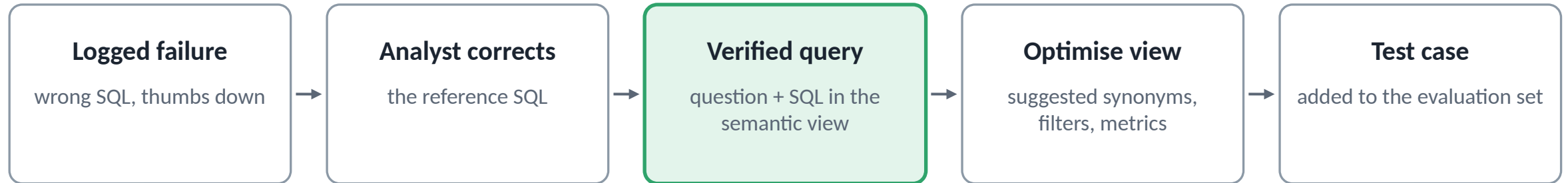
Top-k

fewer, better chunks

Source coverage

abstains reveal missing documents

Failed analytical questions become verified queries



Analytical route

Corrected SQL becomes a verified query. Snowflake can suggest candidate questions from query history and optimise the semantic view from verified queries.

Document route

Repeated abstains on a topic usually mean a missing or stale document: a dbt and ingestion fix, not a prompt fix.

Choose the cheapest model that passes the evaluation

Step	Candidates to compare	Decide on
Route	claude-haiku-4-5, llama3.1-8b	routing accuracy, latency
Answer	claude-sonnet-4-5, openai-gpt-5-mini, llama3.3-70b	faithfulness, cost per answer
Judge	claude-opus-5-5, openai-gpt-5.5	agreement with human labels

Model strings from Snowflake's list at the time of writing; availability varies by region.

Decision rule

1. Set a quality bar per step.
2. Drop models below the bar.
3. Of the rest, pick the cheapest that meets the latency target.

Fine-tune to make a small model behave like a large one

SNOWFLAKE SQL

```
-- training pairs: highly rated answers from the large model
create or replace table ft.train as
select t.standalone_q || '\n\nContext:\n' || c.context
       as prompt,
       t.answer as completion
from obs.chat_turns t
join obs.feedback f      using (turn_id)
join ft.turn_context c    using (turn_id)
where f.rating = 1 and t.route = 'DOCUMENT';

select snowflake.cortex.finetune('CREATE',
    'neonexus_answer_8b',          -- new model
    'llama3.1-8b',                 -- base model
    'select prompt, completion from ft.train',
    'select prompt, completion from ft.validation');

-- then: ai_complete('neonexus_answer_8b', prompt)
```

Good reasons

Cut cost or latency on a narrow task; enforce a consistent format or tone.

Bad reasons

Teaching the model new facts (use retrieval); fixing a problem you have not diagnosed.

Optimise cost and speed without touching quality



Tier the models

small for routing and rewriting, mid for answers, large only for judging



Cap the context

top-k of 3 to 5 good chunks beats 20 average ones



Cache repeat answers

key on standalone question + index version

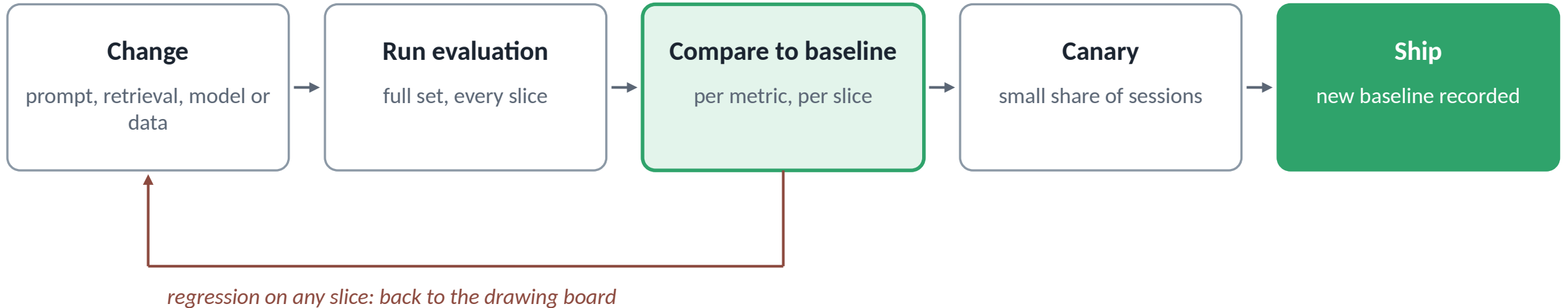


Stream responses

same total time, far better perceived speed

Monitor: CORTEX_AI_FUNCTIONS_USAGE_HISTORY and CORTEX_SEARCH_SERVING_USAGE_HISTORY, joined to obs.chat_turns.

No change ships without passing the regression gate



Example gate policy: no slice drops more than 2 points on faithfulness or recall@5, adversarial pass rate stays at 100%, and cost per answer does not rise more than 10%.

Close the loop on the Neo Nexus assistant

- | | | |
|---|--|--------|
| 1 | Instrument
create obs tables; add logging and feedback to the app | 15 min |
| 2 | Converse
role-play 30 turns as CRM analysts; rate each answer | 15 min |
| 3 | Evaluate
build the eval set; run the judge; check it on 10 answers | 20 min |
| 4 | Diagnose
classify thumbs-down turns; read samples per class | 15 min |
| 5 | Improve
pull one lever; rerun; pass or fail the gate | 25 min |

Deliverables

- obs.chat_turns with at least 30 logged turns
- Judge agreement with your own labels on 10 answers
- Failure breakdown by type
- One change: rung, hypothesis, before and after scores, gate decision

Stretch: add standalone-question rewriting and measure its effect on follow-up questions

Questions worth arguing about

1. Who judges the judge? How much human labelling is enough to trust an LLM grader?
2. When the evaluation set becomes the target, how do you stop overfitting to it?
3. How long should conversation logs be kept, and who may read them?
4. Which single metric would you put on the team's dashboard, and why only one?



References

Zheng et al. (2023) Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS.

Es et al. (2023) RAGAS: Automated Evaluation of RAG.

Shankar et al. (2024) Who Validates the Validators? UIST.

Huyen (2025) AI Engineering. O'Reilly.

Snowflake docs: AI Observability; Cortex Fine-tuning

Snowflake docs: Cortex Analyst verified queries and optimisation; AI cost management

Streamlit and its siblings



Streamlit basics

The script-reruns-on-every-interaction model, widgets and state



Streamlit in Snowflake

Apps that run next to the data, under Snowflake roles



The family

Gradio, Dash, Shiny, Databricks Apps: when to pick which

Before Module 4: keep your obs tables and evaluation set; next time the thumbs and corrections get a proper user interface.